

AWS

AWS

Partie 1 – Introduction au Cloud Computing

Partie 1 – Introduction au Cloud Computing

1. Pourquoi le Cloud Computing existe-t-il ?

- Les limites d'une infrastructure informatique traditionnelle
- Serveurs physiques et datacenters
- Virtualisation
- Besoin de flexibilité et de scalabilité

Les limites d'une infrastructure informatique traditionnelle

Avant le Cloud, une entreprise devait généralement **acheter, installer et maintenir ses propres serveurs** dans ses locaux ou dans un datacenter.

Cela implique plusieurs contraintes :

- **Investissement initial important** : achat des serveurs, du stockage, du réseau et des licences.
- **Délais de mise en place** : commander, recevoir, installer et configurer du matériel peut prendre du temps.
- **Capacité difficile à prévoir** : il faut dimensionner l'infrastructure avant de connaître précisément les besoins futurs.
- **Risque de sous-utilisation** : des serveurs achetés pour absorber les pics de charge peuvent rester peu utilisés le reste du temps.
- **Évolution plus complexe** : augmenter la capacité nécessite souvent d'acheter et d'installer de nouveaux équipements.
- **Maintenance à assurer** : alimentation électrique, refroidissement, mises à jour, remplacement du matériel et gestion des pannes.

Problème principal : l'entreprise doit souvent investir à l'avance dans une capacité qu'elle n'utilisera peut-être pas entièrement.

Serveurs physiques et datacenters

Un **serveur physique** est une machine informatique conçue pour fournir des ressources ou des services à d'autres machines : calcul, stockage, base de données, hébergement d'applications, etc.

Les entreprises regroupent généralement leurs serveurs dans un **datacenter** : un site spécialement conçu pour héberger et faire fonctionner de nombreux équipements informatiques.

Un datacenter doit notamment fournir :

- **Alimentation électrique** fiable et redondante ;
- **Refroidissement** pour éviter la surchauffe des équipements ;
- **Connexion réseau** performante ;
- **Sécurité physique** des locaux ;
- **Surveillance et maintenance** du matériel ;
- **Systèmes de secours** en cas de panne.

Dans une infrastructure traditionnelle, l'entreprise doit donc gérer non seulement ses applications, mais aussi une grande partie de **l'infrastructure physique nécessaire à leur fonctionnement**.

À retenir : une application informatique repose toujours sur du matériel physique quelque part. Le Cloud ne supprime pas les serveurs : il permet surtout de déléguer leur gestion à un fournisseur spécialisé.

La virtualisation

Un serveur physique possède des ressources : **processeur, mémoire, stockage et réseau**.

La **virtualisation** permet de partager ces ressources entre plusieurs **machines virtuelles (VM)** grâce à un logiciel appelé **hyperviseur**.

Chaque VM fonctionne comme un ordinateur indépendant avec :

- son propre système d'exploitation ;
- ses propres ressources virtuelles ;
- ses propres applications.

Cela permet de **mieux utiliser un serveur physique** : au lieu d'exécuter une seule application sur une machine, plusieurs environnements peuvent partager le même matériel.

Et Docker ?

Docker utilise une approche différente : les **conteneurs** partagent le système d'exploitation de la machine hôte et embarquent principalement l'application et ses dépendances.

Ils sont généralement **plus légers et plus rapides à démarrer** qu'une machine virtuelle.

À retenir : la virtualisation permet de mieux exploiter les ressources d'un serveur physique en le partageant entre plusieurs machines virtuelles. C'est une technologie fondamentale du Cloud, car elle permet aux fournisseurs comme AWS de mutualiser leur infrastructure et de proposer des ressources informatiques à la demande. Les conteneurs, comme Docker, vont encore plus loin en isolant les applications avec moins de ressources.

Besoin de flexibilité et de scalabilité

Les besoins informatiques d'une entreprise ne sont pas toujours constants.

Une application peut par exemple recevoir :

- peu d'utilisateurs en temps normal ;
- beaucoup plus de trafic lors d'un lancement, d'une promotion ou d'un événement ;
- une croissance progressive au fil du temps.

Avec une infrastructure physique traditionnelle, augmenter la capacité demande souvent **d'acheter, d'installer et de configurer de nouveaux serveurs.**

Le Cloud apporte davantage de **flexibilité** : les ressources peuvent être adaptées plus rapidement selon les besoins.

La **scalabilité** désigne la capacité d'un système à augmenter ou réduire ses ressources pour supporter une variation de charge.

À retenir : ce besoin d'adapter rapidement les ressources est l'une des raisons majeures de l'existence du Cloud Computing. Plutôt que de dimensionner une infrastructure physique longtemps à l'avance, le Cloud permet d'ajuster plus facilement la capacité en fonction de la demande.

Donc pourquoi le Cloud Computing existe-t-il ?

Le Cloud Computing répond aux limites des infrastructures informatiques traditionnelles. Plutôt que de devoir :

- acheter et maintenir ses propres **serveurs physiques** ;
- gérer soi-même un **datacenter** ;
- prévoir longtemps à l'avance la capacité nécessaire ;
- conserver du matériel parfois sous-utilisé ;
- ajouter manuellement de nouveaux serveurs lorsque les besoins augmentent ;

le Cloud permet de s'appuyer sur l'infrastructure d'un fournisseur spécialisé.

Grâce notamment à la **virtualisation**, les ressources physiques peuvent être mutualisées et proposées sous forme de ressources informatiques utilisables à la demande.

Cela apporte davantage de **flexibilité** et de **scalabilité** : l'infrastructure peut évoluer plus facilement en fonction des besoins.

En résumé : le Cloud Computing existe pour permettre aux entreprises d'utiliser des ressources informatiques sans avoir à posséder et gérer toute l'infrastructure physique nécessaire.

Partie 1 – Introduction au Cloud Computing

2. Qu'est-ce que le Cloud Computing ?

- Définition du Cloud
- Principe général de fonctionnement
- Ressources informatiques à la demande
- Paiement à l'usage

Définition du Cloud

Le **Cloud Computing** consiste à utiliser des ressources informatiques accessibles à distance, généralement via Internet, sans avoir à posséder ni gérer directement toute l'infrastructure physique qui les fait fonctionner.

Ces ressources peuvent être par exemple :

- de la **puissance de calcul** ;
- du **stockage** ;
- des **bases de données** ;
- du **réseau** ;
- des **applications** ;
- différents services de sécurité, de supervision ou d'analyse.

Au lieu d'acheter ses propres serveurs, une entreprise peut utiliser les ressources mises à disposition par un **fournisseur de Cloud** comme AWS, Microsoft Azure ou Google Cloud.

Ces ressources sont généralement disponibles **à la demande** et peuvent être adaptées selon les besoins.

À retenir : le Cloud ne désigne pas un endroit abstrait ou « magique ». Il s'agit toujours de serveurs physiques situés dans des datacenters, mais leur infrastructure est exploitée par un fournisseur qui met des ressources informatiques à disposition de ses clients.

Principe général de fonctionnement

Le Cloud repose sur une idée simple : **les ressources informatiques sont hébergées chez un fournisseur et accessibles à distance par les clients.**

Le fonctionnement général peut se résumer ainsi :

- le fournisseur Cloud possède et exploite des **datacenters** ;
- ces datacenters contiennent de nombreux **serveurs physiques** ;
- grâce notamment à la **virtualisation**, leurs ressources peuvent être découpées et mutualisées ;
- le client demande les ressources dont il a besoin ;
- ces ressources lui sont alors fournies sous forme de services accessibles à distance.

Le client ne gère donc pas directement le matériel physique : il utilise une **couche de services** mise à disposition par le fournisseur Cloud.

À retenir : le Cloud transforme une infrastructure physique complexe en ressources informatiques accessibles comme des services. C'est ce qui permet ensuite de les obtenir rapidement, sans acheter soi-même les serveurs nécessaires.

Des ressources informatiques à la demande

Dans le Cloud, une entreprise n'a pas besoin de préparer toute son infrastructure plusieurs semaines ou plusieurs mois à l'avance.

Elle peut demander des ressources lorsqu'elle en a besoin, par exemple :

- créer un **serveur virtuel** ;
- augmenter une capacité de **stockage** ;
- déployer une **base de données** ;
- ajouter de nouvelles ressources pour absorber une hausse de trafic.

Ces ressources peuvent souvent être provisionnées en quelques minutes depuis une interface, une ligne de commande ou du code.

L'entreprise peut ainsi faire évoluer son infrastructure plus rapidement en fonction de ses besoins réels.

À retenir : “à la demande” signifie que les ressources Cloud peuvent être créées, modifiées ou supprimées lorsque le besoin apparaît. C'est ce fonctionnement qui apporte une grande partie de la flexibilité évoquée précédemment.

Le paiement à l'usage

Dans une infrastructure traditionnelle, une entreprise doit généralement **acheter ses serveurs à l'avance**, même si elle n'utilise pas toute leur capacité.

Dans le Cloud, le modèle économique repose souvent sur le **paiement à l'usage** (*pay-as-you-go*).

Cela signifie que le coût dépend de la consommation réelle de ressources, par exemple :

- durée d'utilisation d'une machine virtuelle ;
- quantité de stockage utilisée ;
- volume de données transférées ;
- nombre de requêtes ou d'opérations effectuées.

Ce modèle permet d'éviter une partie des investissements initiaux importants et d'adapter les dépenses aux besoins.

Cependant, une ressource oubliée ou mal dimensionnée peut continuer à générer des coûts.

À retenir : le Cloud permet de transformer une partie des dépenses informatiques fixes en dépenses liées à l'utilisation réelle. Cette flexibilité financière est un avantage important, mais elle nécessite aussi de surveiller sa consommation.

Partie 1 – Introduction au Cloud Computing

3. À quoi sert le Cloud ?

- Héberger des applications
- Stocker des données
- Exécuter des traitements
- Déployer rapidement une infrastructure
- Adapter les ressources à la demande

Héberger des applications

L'un des usages les plus courants du Cloud est **d'héberger des applications** afin de les rendre accessibles aux utilisateurs via Internet. Il peut s'agir par exemple :

- d'un **site web** ;
- d'une **API** ;
- d'une application métier interne ;
- d'une application e-commerce ;
- d'un backend pour une application mobile.

Au lieu d'acheter et d'administrer ses propres serveurs, l'entreprise peut utiliser les ressources proposées par un fournisseur Cloud.

Quelques exemples de services de calcul :

- **AWS** : Amazon EC2 ;
- **Microsoft Azure** : Azure Virtual Machines ;
- **Google Cloud** : Compute Engine.

Ces services permettent notamment de créer des **machines virtuelles** sur lesquelles les applications peuvent être installées et exécutées.

À retenir : le Cloud permet d'héberger une application sur une infrastructure distante sans avoir à posséder le serveur physique qui l'exécute. On retrouve ici directement les notions de virtualisation et de ressources à la demande vues précédemment.

Stocker des données

Le Cloud permet de **stocker des données à distance** sans avoir à gérer soi-même les disques, baies de stockage ou serveurs nécessaires.

On peut y stocker par exemple :

- des fichiers ;
- des images et vidéos ;
- des sauvegardes ;
- des documents ;
- des données utilisées par une application.

Le stockage Cloud peut être adapté à différents besoins : capacité importante, haute disponibilité, archivage ou accès fréquent aux données.

Quelques exemples de services :

- **AWS** : Amazon S3 ;
- **Microsoft Azure** : Azure Blob Storage ;
- **Google Cloud** : Cloud Storage.

À retenir : le Cloud permet de consommer du stockage comme un service. L'entreprise utilise l'espace dont elle a besoin sans avoir à acheter et maintenir elle-même l'infrastructure physique correspondante.

Exécuter des traitements

Le Cloud ne sert pas uniquement à héberger des applications ou stocker des fichiers : il peut aussi fournir de la **puissance de calcul pour exécuter des traitements**.

Il peut s'agir par exemple de :

- traiter un grand volume de données ;
- convertir ou redimensionner des images ;
- générer des rapports ;
- exécuter des tâches automatisées ;
- effectuer des calculs ponctuels ou intensifs.

Ces traitements peuvent être exécutés sur des machines virtuelles ou grâce à des services qui lancent directement du code à la demande.

Quelques exemples :

- **AWS** : Amazon EC2 ou AWS Lambda ;
- **Microsoft Azure** : Azure Virtual Machines ou Azure Functions ;
- **Google Cloud** : Compute Engine ou Cloud Run functions.

À retenir : le Cloud permet d'obtenir de la puissance de calcul lorsque cela est nécessaire, puis de réduire ou supprimer les ressources une fois le traitement terminé. On retrouve ici les notions de ressources à la demande et de scalabilité.

Déployer rapidement une infrastructure

Le Cloud permet de **créer une infrastructure informatique beaucoup plus rapidement** qu'avec du matériel physique traditionnel.

Au lieu de devoir commander, installer et configurer de nouveaux serveurs, une entreprise peut créer des ressources en quelques minutes depuis :

- une console web ;
- une ligne de commande ;
- du code ou des outils d'automatisation.

Il devient ainsi possible de déployer rapidement :

- des serveurs ;
- du stockage ;
- des bases de données ;
- des réseaux ;
- des environnements de test ou de développement.

À retenir : le Cloud réduit fortement le temps nécessaire pour passer d'un besoin à une infrastructure utilisable. C'est un avantage majeur pour tester, déployer et faire évoluer rapidement des projets.

Adapter les ressources à la demande

Les besoins d'une application peuvent varier dans le temps.

Le Cloud permet d'**augmenter ou de réduire les ressources** en fonction de la charge réelle.

Par exemple :

- ajouter de la puissance de calcul lors d'un pic de trafic ;
- augmenter la capacité de stockage lorsque les données augmentent ;
- réduire les ressources lorsque l'activité diminue ;
- créer temporairement des ressources pour un besoin ponctuel.

Cette capacité d'adaptation correspond à la notion de **scalabilité** vue précédemment.

À retenir : le Cloud permet d'adapter l'infrastructure aux besoins réels au lieu de conserver en permanence une capacité maximale. On utilise davantage de ressources quand elles sont nécessaires, et moins lorsqu'elles ne le sont plus.

Conclusion a quoi sert le Cloud ?

Le Cloud permet notamment de :

- héberger des applications ;
- stocker des données ;
- exécuter des traitements ;
- déployer rapidement une infrastructure ;
- adapter les ressources en fonction des besoins.

Ces exemples représentent les **usages les plus courants et les plus fondamentaux**, mais ils ne couvrent qu'une partie des possibilités offertes par le Cloud.

Les fournisseurs Cloud proposent aujourd'hui des centaines de services permettant également de travailler sur :

- l'intelligence artificielle ;
- l'analyse de données ;
- la cybersécurité ;
- l'Internet des objets (IoT) ;
- la diffusion de contenu ;
- l'automatisation et bien d'autres domaines.

À retenir : le Cloud n'est pas un service unique, mais un vaste ensemble de ressources et de services informatiques accessibles à la demande. Les exemples vus jusqu'ici constituent les briques de base qui permettront de comprendre les services plus spécialisés par la suite.

Partie 1 – Introduction au Cloud Computing

4. Les principaux modèles de Cloud

- Cloud public
- Cloud privé
- Cloud hybride

Cloud public vs Cloud privé

Cloud public

Le **Cloud public** repose sur une infrastructure exploitée par un fournisseur comme **AWS**, **Microsoft Azure** ou **Google Cloud**.

Les ressources physiques sont mutualisées entre plusieurs clients, tout en restant isolées logiquement.

Avantages

- mise en place rapide ;
- grande flexibilité ;
- paiement à l'usage ;
- pas de datacenter à gérer soi-même.

Cloud public vs Cloud privé

Cloud privé

Le **Cloud privé** est une infrastructure Cloud dédiée à une seule organisation.

Elle peut être hébergée dans les propres datacenters de l'entreprise ou exploitée par un prestataire.

Avantages

- contrôle plus important sur l'infrastructure ;
- personnalisation plus poussée ;
- adapté à certains besoins spécifiques de sécurité, conformité ou gouvernance.

À retenir : dans un Cloud public, l'infrastructure physique est partagée entre plusieurs clients ; dans un Cloud privé, elle est dédiée à une seule organisation.

Cloud hybride

Le **Cloud hybride** combine une infrastructure privée avec des services de Cloud public.

Une entreprise peut ainsi conserver certaines ressources dans son propre environnement tout en utilisant AWS, Azure ou Google Cloud pour d'autres besoins.

Par exemple :

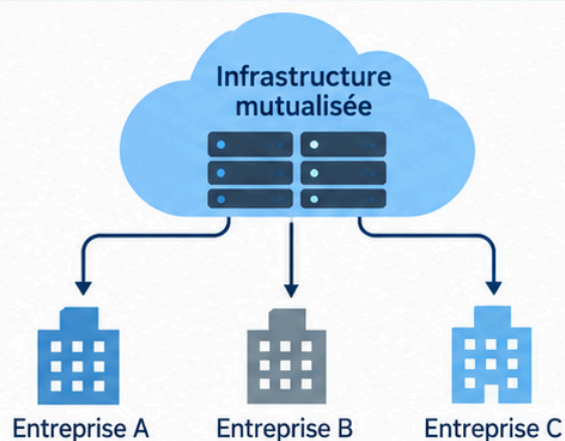
- conserver des données sensibles dans une infrastructure privée ;
- utiliser le Cloud public pour absorber un pic de trafic ;
- héberger certaines applications en interne et d'autres dans le Cloud ;
- utiliser des services Cloud spécialisés sans migrer toute l'infrastructure.

Le Cloud hybride permet donc de **combiner contrôle et flexibilité**.

À retenir : le Cloud hybride relie un environnement privé et un Cloud public afin d'utiliser chaque modèle là où il est le plus adapté.

Les principaux modèles de Cloud

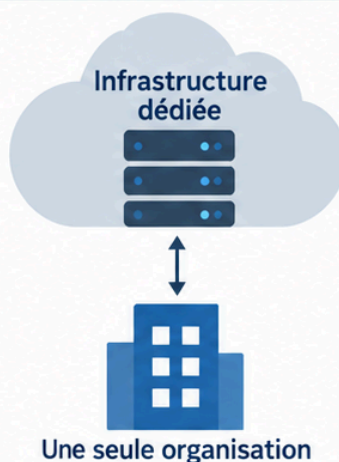
Cloud public



- Infrastructure exploitée par un fournisseur
- Ressources mutualisées entre plusieurs clients
- Accessible via Internet

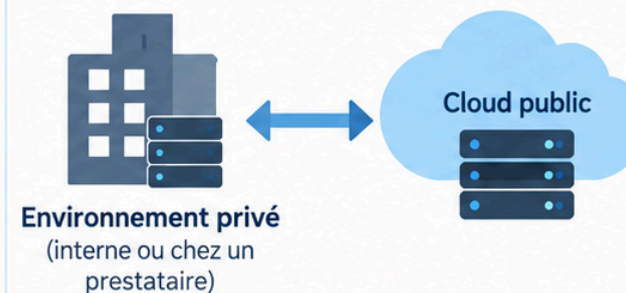
Exemples : AWS, Azure, Google Cloud

Cloud privé



- Infrastructure dédiée à une seule organisation
- Peut être hébergée en interne ou chez un prestataire
- Contrôle plus important

Cloud hybride



- Combine Cloud privé et Cloud public
- Certaines ressources restent en interne
- Permet de combiner contrôle et flexibilité



En résumé : public = partagé, privé = dédié, hybride = combinaison des deux.

Partie 1 – Introduction au Cloud Computing

5. Les principaux modèles de services

- IaaS
- PaaS
- SaaS

Ces trois modèles permettent de comprendre **jusqu'où le fournisseur Cloud prend en charge l'infrastructure et à partir de quel niveau le client reste responsable**. Cette distinction sera importante pour comprendre les services AWS, leur utilisation et leur niveau de gestion.

IaaS – Infrastructure as a Service

Le modèle **IaaS** (*Infrastructure as a Service*) consiste à louer des ressources d'infrastructure informatique auprès d'un fournisseur Cloud.

Le fournisseur gère notamment :

- les datacenters ;
- les serveurs physiques ;
- le stockage physique ;
- le réseau physique ;
- la couche de virtualisation.

Le client garde la responsabilité de ce qu'il installe au-dessus, par exemple :

- le système d'exploitation ;
- les applications ;
- une partie de la configuration et de la sécurité.

Exemples

- **AWS** : Amazon EC2 ;
- **Microsoft Azure** : Azure Virtual Machines ;
- **Google Cloud** : Compute Engine.

À retenir : avec l'IaaS, on ne gère plus le matériel physique, mais on conserve encore beaucoup de contrôle sur la machine virtuelle et sur ce qui y est installé.

PaaS – Platform as a Service

Le modèle **PaaS** (*Platform as a Service*) fournit une plateforme prête à l'emploi pour développer, déployer et exécuter des applications.

Le fournisseur Cloud gère davantage d'éléments que dans l'IaaS, notamment :

- l'infrastructure physique ;
- la virtualisation ;
- le système d'exploitation ;
- une partie de l'environnement d'exécution.

Le client se concentre principalement sur :

- son code ;
- son application ;
- ses données ;
- sa configuration fonctionnelle.

Exemples

- **AWS** : AWS Elastic Beanstalk ;
- **Microsoft Azure** : Azure App Service ;
- **Google Cloud** : App Engine.

À retenir : avec le PaaS, le client gère moins d'infrastructure qu'en IaaS et peut se concentrer davantage sur le développement et le fonctionnement de son application.

SaaS — Software as a Service

Le modèle **SaaS** (*Software as a Service*) consiste à utiliser directement une application hébergée et gérée par un fournisseur.

Le fournisseur prend en charge presque toute la partie technique :

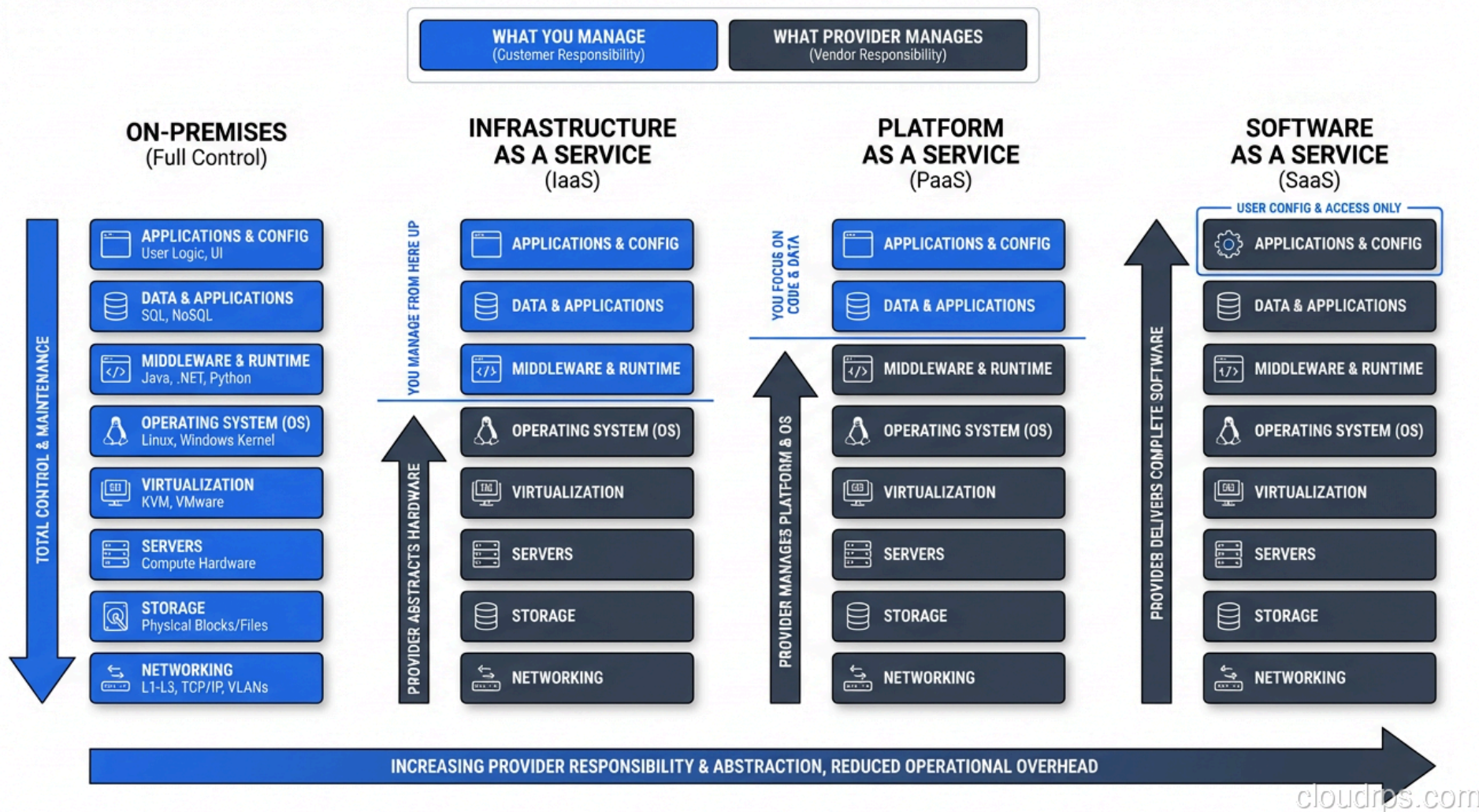
- l'infrastructure ;
- les serveurs ;
- le système d'exploitation ;
- l'application ;
- les mises à jour ;
- une grande partie de la maintenance.

Le client utilise principalement le logiciel et configure ce qui concerne son propre usage.

Exemples

- Microsoft 365 ;
- Google Workspace ;
- Salesforce.

À retenir : avec le SaaS, le client ne gère pratiquement plus l'infrastructure ni l'application elle-même : il consomme directement un logiciel prêt à l'emploi.



IaaS, PaaS et SaaS – À retenir

Les modèles IaaS, PaaS et SaaS représentent différents **niveaux de responsabilité entre le client et le fournisseur Cloud**.

- **IaaS** → davantage de contrôle, mais davantage d'éléments à administrer.
- **PaaS** → le fournisseur gère une plus grande partie de la plateforme.
- **SaaS** → le client utilise directement une application presque entièrement gérée par le fournisseur.

Plus on se rapproche du SaaS, plus le fournisseur prend en charge de composants techniques et de tâches de maintenance.

Concernant les coûts : un service davantage managé peut être facturé différemment ou coûter davantage à consommation comparable, mais il peut également réduire les coûts d'administration et de maintenance.

Partie 1 – Introduction au Cloud Computing

6. Avantages et limites du Cloud

- Flexibilité
- Scalabilité
- Haute disponibilité
- Rapidité de déploiement
- Coûts
- Sécurité
- Dépendance au fournisseur

Les principaux avantages du Cloud - Rappel

Le Cloud apporte plusieurs avantages par rapport à une infrastructure traditionnelle :

- **Flexibilité** : créer, modifier ou supprimer rapidement des ressources selon les besoins.
- **Scalabilité** : augmenter ou réduire la capacité d'une infrastructure en fonction de la charge.
- **Rapidité de déploiement** : mettre en place des serveurs, bases de données ou services en quelques minutes.
- **Haute disponibilité** : répartir une application sur plusieurs infrastructures afin de limiter l'impact d'une panne.
- **Accès à de nombreux services** : utiliser des technologies avancées sans devoir construire toute l'infrastructure soi-même.

Ces avantages permettent aux entreprises de **déployer plus vite, d'expérimenter plus facilement et d'adapter leur infrastructure à leur activité.**

À retenir : le principal intérêt du Cloud est de rendre l'infrastructure informatique plus souple et plus rapide à faire évoluer. Cela ne supprime pas les contraintes techniques, mais permet de les gérer différemment.

Les limites et points de vigilance du Cloud

Le Cloud apporte de nombreux avantages, mais il ne supprime pas toutes les contraintes.

Il faut notamment surveiller :

- **Les coûts** : une mauvaise configuration, un surdimensionnement ou des ressources oubliées peuvent générer des dépenses importantes.
- **La sécurité** : le fournisseur sécurise son infrastructure, mais le client reste responsable d'une partie de la configuration, des accès et des données.
- **La dépendance au fournisseur** : plus une architecture utilise des services spécifiques à un Cloud, plus une migration vers un autre fournisseur peut devenir complexe.
- **Les compétences nécessaires** : utiliser correctement le Cloud demande de comprendre les services, leur configuration et leurs bonnes pratiques.
- **La complexité** : une architecture Cloud mal conçue peut devenir difficile à maintenir malgré la simplicité apparente de création des ressources.

À retenir : le Cloud ne signifie pas « plus de contraintes », mais plutôt **des responsabilités différentes**. Il faut toujours surveiller les coûts, la sécurité et la conception de l'infrastructure.

Cloud : avantages et responsabilités

Le Cloud permet de gagner en **rapidité, flexibilité et capacité d'adaptation**, mais ces avantages ne dispensent pas d'une bonne conception.

Une utilisation efficace du Cloud nécessite notamment de :

- choisir des services adaptés au besoin ;
- dimensionner correctement les ressources ;
- surveiller les coûts ;
- sécuriser les accès et les données ;
- prévoir les pannes et la disponibilité ;
- supprimer les ressources devenues inutiles.

Le fournisseur Cloud prend en charge une partie importante de l'infrastructure, mais **le client conserve toujours certaines responsabilités**.

À retenir : le Cloud simplifie de nombreux aspects de l'infrastructure, mais il ne remplace ni les bonnes pratiques, ni la sécurité, ni la maîtrise des coûts.

Partie 1 – Introduction au Cloud Computing

7. Les principaux fournisseurs de Cloud

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)

AWS, Microsoft Azure et Google Cloud sont aujourd'hui les **trois principaux fournisseurs mondiaux de services Cloud**. Ils concentrent à eux trois la majorité du marché mondial des services d'infrastructure Cloud.

Il existe cependant de nombreux autres acteurs, comme **Oracle Cloud (OCI)**, **IBM Cloud** ou encore **Alibaba Cloud**.

L'objectif ici est de présenter brièvement les trois acteurs majeurs.

Amazon Web Services (AWS)

- **Société** : Amazon
- **Lancement** : 2006
- **Positionnement** : plateforme Cloud généraliste proposant un très large catalogue de services.
- **Services connus** : Amazon EC2, Amazon S3, Amazon RDS.

Points forts

- catalogue de services très étendu ;
- forte maturité de la plateforme ;
- infrastructure mondiale importante ;
- nombreuses possibilités d'architecture et d'automatisation.

Points d'attention

- grand nombre de services pouvant rendre la prise en main complexe ;
- tarification parfois difficile à anticiper sans surveillance.

À retenir : AWS est l'un des pionniers du Cloud moderne.

Microsoft Azure

- **Société** : Microsoft
- **Lancement** : 2010
- **Positionnement** : plateforme Cloud généraliste fortement intégrée à l'écosystème Microsoft.
- **Services connus** : Azure Virtual Machines, Azure Blob Storage, Azure SQL.

Points forts

- forte intégration avec les technologies Microsoft ;
- adapté aux environnements d'entreprise utilisant déjà Microsoft ;
- offre importante en Cloud hybride ;
- large catalogue de services.

Points d'attention

- catalogue et tarification pouvant devenir complexes ;
- certaines solutions sont particulièrement intéressantes lorsqu'on utilise déjà l'écosystème Microsoft.

À retenir : Azure est un concurrent majeur d'AWS, particulièrement présent dans les environnements d'entreprise.

Google Cloud

- **Société** : Google
- **Premiers services Cloud** : 2008 avec Google App Engine
- **Positionnement** : plateforme Cloud généraliste s'appuyant fortement sur l'expertise technique de Google.
- **Services connus** : Compute Engine, Cloud Storage, BigQuery.

Points forts

- forte expertise dans la donnée et l'analyse ;
- services reconnus autour de l'IA et du Machine Learning ;
- technologies Cloud et conteneurs très présentes dans son écosystème.

Points d'attention

- part de marché historiquement plus faible qu'AWS et Azure ;
- écosystème d'entreprise moins ancien que celui de ses deux principaux concurrents.

À retenir : Google Cloud se distingue notamment par son expertise autour de la donnée, de l'IA et des technologies Cloud modernes.

Conclusion – Introduction au Cloud Computing

À la fin de cette partie, nous avons vu :

- pourquoi le **Cloud Computing** est apparu et à quelles limites il répond.
- le rôle des **serveurs physiques, datacenters et de la virtualisation**.
- le principe des **ressources à la demande** et du **paiement à l'usage**.
- les principaux usages du Cloud : hébergement, stockage, calcul et déploiement d'infrastructures.
- les modèles de déploiement : **Cloud public, privé et hybride**.
- les modèles de services : **IaaS, PaaS et SaaS**.
- les principaux avantages, limites et points de vigilance du Cloud.
- les principaux fournisseurs : **AWS, Microsoft Azure et Google Cloud**.

À retenir : le Cloud permet de consommer des ressources et des services informatiques de manière flexible, sans avoir à gérer soi-même toute l'infrastructure physique nécessaire.

Partie 2 — Introduction à Amazon Web Services

Partie 2 – Introduction à Amazon Web Services

1. Qu'est-ce qu'AWS ?

- Présentation d'Amazon Web Services
- Rôle d'AWS dans le Cloud
- Organisation générale de la plateforme

Présentation d'Amazon Web Services

Amazon Web Services (AWS) est la plateforme de Cloud Computing proposée par Amazon.

Lancée en **2006**, elle permet aux entreprises, développeurs et organisations d'accéder à des ressources informatiques à la demande via Internet.

AWS propose notamment des services pour :

- le **calcul** ;
- le **stockage** ;
- les **bases de données** ;
- le **réseau** ;
- la **sécurité** ;
- l'analytique et l'**intelligence artificielle**.

Comme vu précédemment avec le Cloud Computing, ces ressources peuvent être provisionnées rapidement et sont généralement facturées selon leur utilisation.

AWS propose aujourd'hui **plus de 240 services** couvrant de très nombreux besoins informatiques.

À retenir : AWS est une plateforme Cloud complète qui permet de construire et d'exploiter une infrastructure informatique sans avoir à posséder soi-même les datacenters et les serveurs physiques sous-jacents.

Quelques repères historiques sur AWS

AWS est né d'un besoin concret rencontré par Amazon : **déployer et gérer une infrastructure informatique était complexe et ralentissait les équipes.**

Quelques repères :

- **2006** : AWS lance ses premiers grands services publics, notamment **Amazon S3, Amazon EC2 et Amazon SQS.**
- Au fil des années, AWS enrichit fortement son catalogue avec des services de bases de données, réseau, sécurité, serverless, analyse de données, IA, etc.
- Aujourd'hui, AWS propose **plusieurs centaines de services** et s'appuie sur une infrastructure mondiale.

À retenir : AWS est passé de quelques services fondamentaux en 2006 à une plateforme Cloud très large couvrant aujourd'hui la majorité des besoins informatiques d'une entreprise.

Anecdote

Lors du lancement d'Amazon EC2 en 2006, AWS ne proposait qu'un seul type d'instance : `m1.small`.

Quelques grandes entreprises utilisant AWS

De nombreuses entreprises et plateformes mondialement connues utilisent AWS pour héberger, traiter ou faire évoluer une partie importante de leurs services.

Quelques exemples :

- **Netflix** : utilise AWS pour une grande partie de son infrastructure, du calcul, du stockage et de ses plateformes de données.
- **Airbnb** : s'appuie sur AWS pour faire évoluer son infrastructure et accompagner sa croissance mondiale.
- **Epic Games / Fortnite** : utilise AWS pour supporter des millions de joueurs et adapter automatiquement ses ressources lors des pics de charge.
- **Formula 1** : utilise AWS pour traiter et analyser d'importants volumes de données en temps réel.

À retenir : ces exemples montrent qu'AWS peut aussi bien répondre aux besoins d'un petit projet qu'à ceux de plateformes mondiales devant gérer de très grandes quantités d'utilisateurs, de données et de trafic.

Le rôle d'AWS dans le Cloud

AWS est un **fournisseur de Cloud public** (rappel) : il met à disposition des ressources et des services informatiques accessibles à distance.

Son rôle est notamment de fournir :

- de la puissance de **calcul** ;
- du **stockage** ;
- des **bases de données** ;
- du **réseau** ;
- des services de **sécurité**, de supervision, d'IA, d'analyse de données, etc.

AWS exploite l'infrastructure physique nécessaire : datacenters, serveurs, réseau et une grande partie des couches techniques sous-jacentes. Le client choisit ensuite les services dont il a besoin et reste responsable de leur **bonne configuration et de leur utilisation**.

À retenir : AWS ne remplace pas le Cloud Computing : **AWS est l'un des fournisseurs qui permettent de consommer concrètement des services Cloud**. Il applique donc les principes vus précédemment : ressources à la demande, scalabilité, paiement à l'usage et services managés.

Organisation générale de la plateforme AWS

AWS est organisé autour de plusieurs éléments principaux :

- un **compte AWS**, qui constitue l'environnement principal du client ;
- des **services AWS**, chacun répondant à un besoin précis : calcul, stockage, réseau, bases de données, sécurité, etc.
- des **ressources**, créées à l'intérieur de ces services : une instance EC2, un bucket S3, une base de données, etc.
- une **infrastructure mondiale**, composée notamment de Regions et d'Availability Zones
- des mécanismes de **sécurité et de permissions** permettant de contrôler qui peut accéder aux ressources.

Les services peuvent ensuite être utilisés depuis la **Console AWS**, AWS CLI, des SDK ou directement via des API.

À retenir : AWS n'est pas une application unique, mais une plateforme composée de nombreux services indépendants qui permettent de créer et d'administrer différentes ressources Cloud au sein d'un compte AWS.

Avant d'aller plus loin

Nous avons vu qu'AWS est une plateforme Cloud composée de nombreux services permettant de créer et gérer des ressources informatiques à la demande.

Pour comprendre correctement le fonctionnement de ces services, il faut maintenant comprendre **où les ressources AWS sont hébergées et comment l'infrastructure mondiale d'AWS est organisée.**

Prochaine étape : Regions, Availability Zones et infrastructure mondiale AWS.

Partie 2 – Introduction à Amazon Web Services

2. L'infrastructure mondiale AWS

- Regions
- Availability Zones
- Edge Locations
- Services régionaux et services globaux

Les Regions AWS

Une **Region AWS** est une zone géographique dans laquelle AWS possède et exploite plusieurs infrastructures de datacenters. Chaque Region :

- correspond à une **zone géographique précise** ;
- est **isolée des autres Regions** ;
- contient plusieurs **Availability Zones (AZ)** ;
- possède un identifiant, par exemple :
 - `eu-west-3` → Paris ;
 - `eu-west-1` → Irlande ;
 - `us-east-1` → Virginie du Nord.

Lorsqu'on crée une ressource AWS, il faut souvent **choisir la Region dans laquelle elle sera déployée**.

Ce choix peut dépendre notamment :

- de la proximité avec les utilisateurs ;
- des exigences de localisation des données ;
- de la disponibilité des services ;
- des coûts.

À retenir : une Region représente une zone géographique AWS. La plupart des ressources sont créées dans une Region précise et n'existent pas automatiquement dans les autres Regions.

Les Availability Zones (AZ)

Une **Availability Zone** est un emplacement d'infrastructure distinct à l'intérieur d'une Region AWS.

Une Region contient plusieurs AZ afin de permettre de répartir les ressources sur plusieurs emplacements indépendants.

Chaque AZ :

- regroupe un ou plusieurs datacenters ;
- dispose de sa propre alimentation, de son propre réseau et de ses propres infrastructures ;
- est reliée aux autres AZ de la même Region par des connexions réseau rapides et à faible latence.

Exemple dans la Region de Paris :

- eu-west-3a
- eu-west-3b
- eu-west-3c

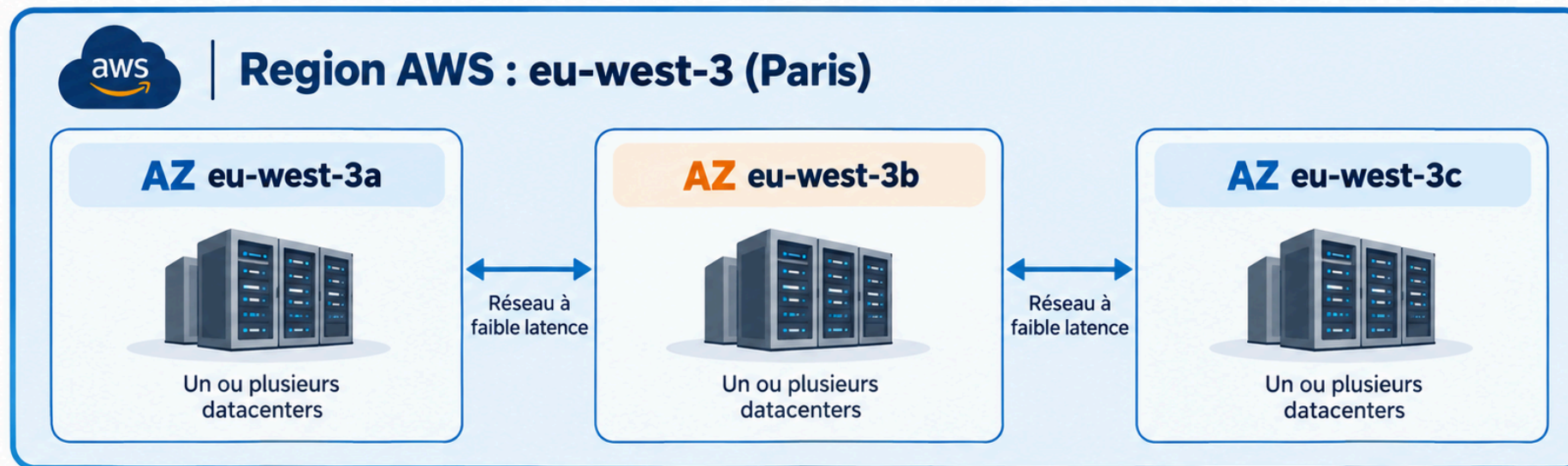
L'intérêt principal est la **haute disponibilité** : une application peut être répartie sur plusieurs AZ afin de continuer à fonctionner même si une zone rencontre un problème.

À retenir : une Region contient plusieurs Availability Zones. Répartir une architecture sur plusieurs AZ permet de réduire la dépendance à un seul emplacement physique.

Regions



Region AWS et Availability Zones



Idée clé

- 1 Une Region contient **plusieurs AZ**
- 2 Chaque AZ est **physiquement distincte**
- 3 Répartir une application sur plusieurs AZ **améliore la disponibilité**



Region = zone géographique

AZ = emplacement distinct dans la Region

Les Edge Locations

Les **Edge Locations** sont des sites d'infrastructure AWS répartis dans de nombreux endroits du monde, au plus proche des utilisateurs finaux.

Leur rôle principal est de **rapprocher certains services AWS des utilisateurs** afin de réduire la latence et d'améliorer les performances.

Elles sont notamment utilisées par :

- **Amazon CloudFront** pour distribuer du contenu plus rapidement ;
- **Amazon Route 53** pour répondre aux requêtes DNS ;
- **AWS Global Accelerator** pour améliorer l'accès à certaines applications.

Contrairement aux Regions, les Edge Locations ne servent pas principalement à héberger directement des ressources comme des machines virtuelles ou des bases de données.

À retenir : une Edge Location est un point de présence AWS proche des utilisateurs. Elle sert surtout à accélérer l'accès aux contenus et services distribués à l'échelle mondiale.

Services régionaux et services globaux

Dans AWS, certains services sont **régionaux** et d'autres sont **globaux**.

Services régionaux

Un service régional fonctionne dans une **Region précise**.

Les ressources créées appartiennent à cette Region.

Exemples

- Amazon EC2
- Amazon RDS
- Amazon VPC
- AWS Lambda

Services globaux

Un service global n'est pas limité à une seule Region, ou fonctionne à l'échelle du compte AWS ou du réseau mondial AWS.

Exemples

- AWS IAM
- Amazon Route 53
- Amazon CloudFront
- AWS Organizations

À **retenir** : la plupart des ressources AWS sont régionales, ce qui signifie qu'il faut choisir leur Region de déploiement. Certains services, en revanche, fonctionnent de manière globale à l'échelle d'AWS.

Attention à la Region sélectionnée

Lorsqu'on travaille avec AWS, une grande partie des ressources sont créées dans une **Region précise**.

Cela peut provoquer une confusion fréquente :

- vous créez une ressource dans `eu-west-3` (Paris)
- vous changez ensuite de Region dans la Console AWS
- la ressource semble avoir "disparu"
- en réalité, elle existe toujours, mais dans la Region où elle a été créée.

Exemple :

Une instance **EC2** créée à Paris ne sera pas visible si la Console AWS est positionnée sur l'Irlande. Il faut donc toujours vérifier la **Region sélectionnée** avant de chercher ou créer une ressource régionale.

À retenir : si une ressource AWS semble introuvable, l'un des premiers réflexes doit être de vérifier la Region active dans la Console.

Conclusion — Infrastructure mondiale AWS

À la fin de cette partie, nous avons vu que l'infrastructure AWS est organisée autour de plusieurs niveaux :

- les **Regions**, qui correspondent à des zones géographiques ;
- les **Availability Zones**, qui sont des emplacements distincts à l'intérieur d'une Region ;
- les **Edge Locations**, utilisées pour rapprocher certains services des utilisateurs ;
- les **services régionaux**, dont les ressources existent dans une Region précise ;
- les **services globaux**, qui fonctionnent à une échelle plus large que celle d'une seule Region.

Cette organisation permet à AWS de proposer des architectures **disponibles, distribuées et adaptées aux besoins géographiques des utilisateurs**.

À retenir : lorsqu'on travaille avec AWS, il faut toujours savoir **dans quelle Region se trouvent les ressources**, comment elles sont réparties entre les Availability Zones, et si le service utilisé est régional ou global.

Partie 2 – Introduction à Amazon Web Services

3. Les grandes familles de services AWS

- Compute
- Storage
- Networking
- Databases
- Security
- Monitoring
- Serverless
- Messaging
- Autres grandes catégories

Les grandes familles de services AWS

AWS propose aujourd'hui **plus de 240 services** couvrant de très nombreux besoins informatiques.

Il serait inutile de chercher à tous les apprendre ou à tous les présenter dès maintenant.

Pour mieux comprendre la plateforme, les services peuvent être regroupés en grandes familles :

- calcul ;
- stockage ;
- bases de données ;
- réseau ;
- sécurité ;
- supervision ;
- intégration et messagerie ;
- intelligence artificielle, analyse de données, IoT, etc.

Le catalogue AWS **évolue en permanence** : de nouveaux services apparaissent, certains sont renommés, regroupés, placés en maintenance ou progressivement retirés.

À retenir : l'objectif n'est pas de connaître tous les services AWS, mais de comprendre les grandes familles et d'identifier progressivement les services fondamentaux.

Calcul, stockage et bases de données

Ces familles regroupent les services permettant de **faire fonctionner des applications et de gérer leurs données**.

Calcul

Permet d'exécuter des applications, des serveurs ou du code.

Exemples : **Amazon EC2** , **AWS Lambda** , **Amazon ECS / EKS**

Stockage

Permet de conserver des fichiers, sauvegardes ou volumes de données.

Exemples : **Amazon S3** , **Amazon EBS** , **Amazon EFS**

Bases de données

Permet de stocker et organiser les données utilisées par les applications.

Exemples : **Amazon RDS** , **Amazon DynamoDB** , **Amazon Aurora**

À retenir : calcul, stockage et bases de données constituent trois des briques fondamentales utilisées pour construire une application dans AWS.

Réseau et sécurité

Ces familles permettent de **connecter les ressources AWS et de contrôler leur accès**.

Réseau

Permet notamment de créer des réseaux, gérer les communications et exposer des applications sur Internet.

Exemples : **Amazon VPC** , **Amazon Route 53** , **Amazon CloudFront**

Sécurité

Permet de gérer les identités, les permissions, les secrets et la protection des ressources.

Exemples : **AWS IAM** , **AWS Secrets Manager** , **AWS WAF**

À retenir : une infrastructure Cloud ne consiste pas seulement à créer des serveurs : il faut également contrôler comment ils communiquent et qui a le droit d'y accéder.

Supervision, intégration et services spécialisés

AWS propose également de nombreuses familles de services permettant d'exploiter et d'enrichir une infrastructure.

Supervision

Observer l'état et le comportement des ressources.

Exemple : **Amazon CloudWatch**

Intégration et messagerie

Permettre aux applications et services de communiquer entre eux.

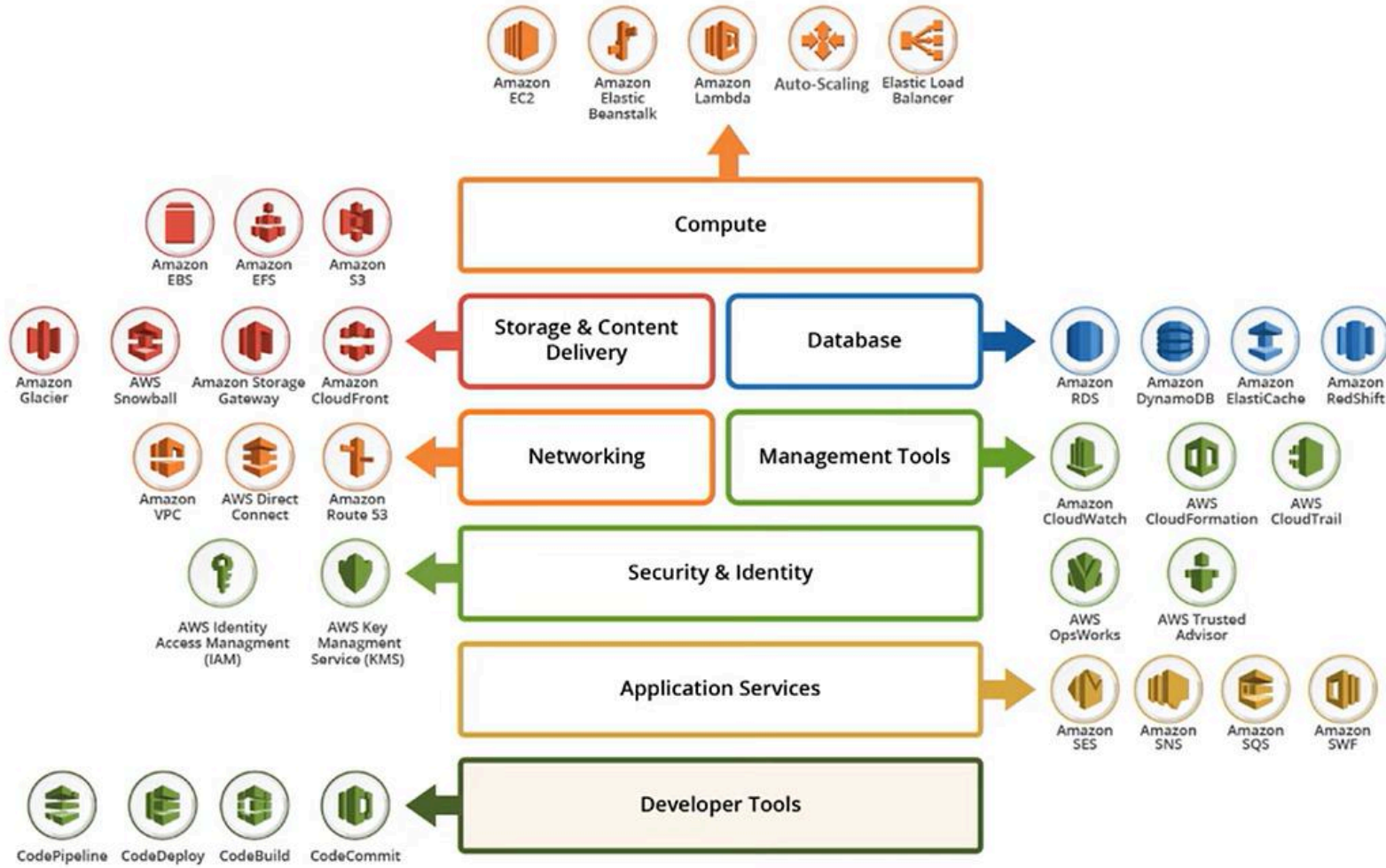
Exemples : **Amazon SQS** , **Amazon SNS** , **Amazon API Gateway**

Services spécialisés

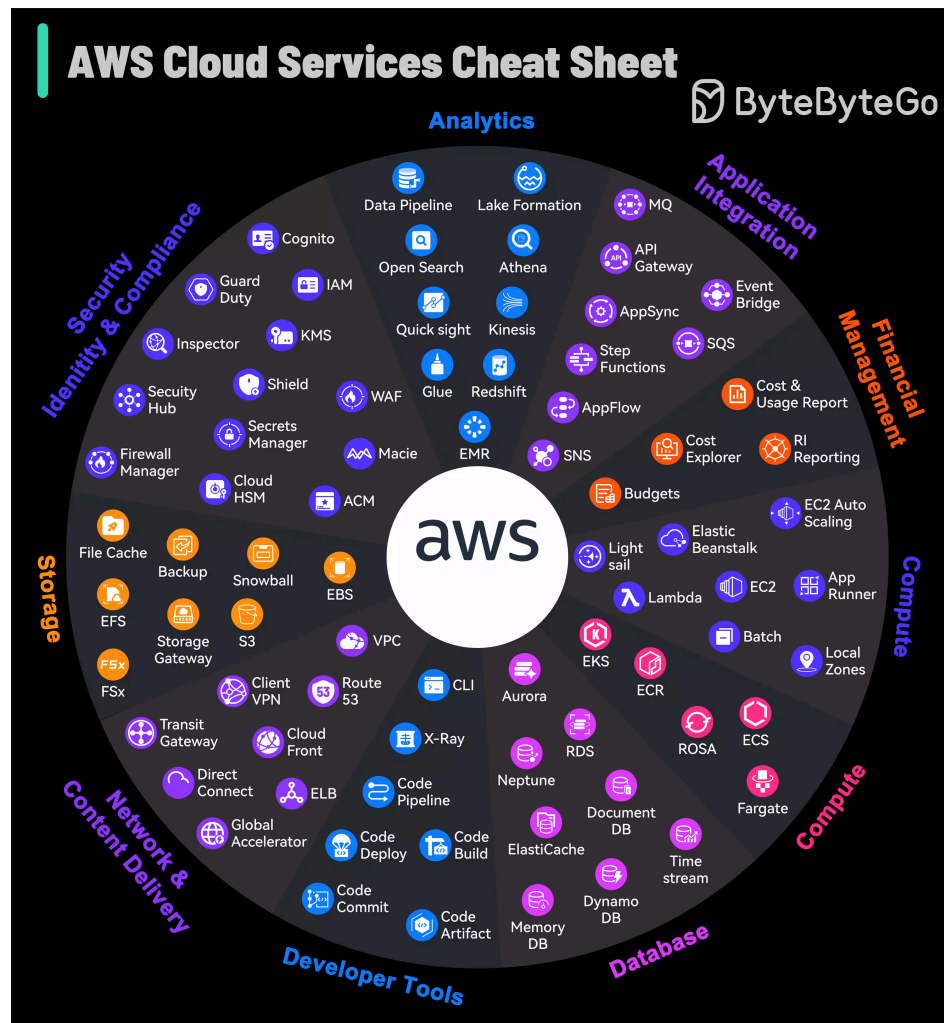
AWS propose également des services dédiés à l'intelligence artificielle et au Machine Learning , l'analyse de données ,l'IoT ,au DevOps et à l'automatisation , la migration et bien d'autres domaines.

À retenir : AWS couvre bien plus que l'hébergement classique. Son catalogue fournit des briques spécialisées permettant de construire des architectures complètes.

Services AWS



Services AWS



Partie 2 – Introduction à Amazon Web Services

4. Quelques services AWS fondamentaux

- AWS IAM
- Amazon EC2
- Amazon S3
- Amazon RDS
- AWS Lambda

AWS IAM => Identity and Access Management

- **Famille** : Sécurité, identité et conformité
- **Portée** : Global
- **Rôle principal** : gérer les identités et contrôler les permissions dans AWS.
- **À quoi sert-il ?** : déterminer qui peut accéder à quelles ressources et quelles actions sont autorisées.
- **Exemple d'utilisation** : autoriser un utilisateur à consulter un bucket S3 sans lui permettre de supprimer des objets.
- **Point fort** : permet de gérer très finement les accès aux ressources AWS.
- **Point d'attention** : une mauvaise configuration des permissions peut donner trop d'accès ou bloquer des utilisateurs légitimes.

À retenir : IAM est le service central de gestion des identités et des permissions dans AWS.

Amazon EC2 => Elastic Compute Cloud

- **Famille** : Calcul
- **Portée** : Régional
- **Rôle principal** : fournir des machines virtuelles dans AWS.
- **À quoi sert-il ?** : exécuter des applications, des serveurs web, des services ou des traitements.
- **Exemple d'utilisation** : créer une machine virtuelle Linux pour héberger une application web.
- **Point fort** : offre beaucoup de contrôle sur le système d'exploitation et les ressources utilisées.
- **Point d'attention** : le client reste responsable d'une grande partie de l'administration de la machine.

À retenir : EC2 permet de louer de la puissance de calcul sous forme de machines virtuelles configurables.

Amazon S3 => Simple Storage Service

- **Famille** : Stockage
- **Portée** : Régional
- **Rôle principal** : stocker des objets dans le Cloud.
- **À quoi sert-il ?** : conserver des fichiers, sauvegardes, images, documents ou données applicatives.
- **Exemple d'utilisation** : stocker les images envoyées par les utilisateurs d'une application.
- **Point fort** : très grande capacité de stockage et forte durabilité des données.
- **Point d'attention** : les permissions d'accès doivent être configurées avec soin pour éviter une exposition involontaire des données.

À retenir : S3 est le service de stockage d'objets de référence dans AWS.

Amazon RDS => Relational Database Service

- **Famille** : Bases de données
- **Portée** : Régional
- **Rôle principal** : fournir des bases de données relationnelles managées.
- **À quoi sert-il ?** : utiliser des moteurs comme PostgreSQL, MySQL ou MariaDB sans gérer toute l'infrastructure de la base de données.
- **Exemple d'utilisation** : héberger la base PostgreSQL utilisée par une application web.
- **Point fort** : AWS automatise une partie importante de l'administration, des sauvegardes et de la maintenance.
- **Point d'attention** : le coût dépend notamment du moteur, de la taille de l'instance, du stockage et des options choisies.

À retenir : RDS permet d'utiliser une base de données relationnelle tout en déléguant une partie de son administration à AWS.

AWS Lambda

- **Famille** : Calcul / Serverless
- **Portée** : Régional
- **Rôle principal** : exécuter du code sans avoir à administrer directement de serveur.
- **À quoi sert-il ?** : lancer automatiquement du code en réponse à un événement ou à une requête.
- **Exemple d'utilisation** : redimensionner automatiquement une image lorsqu'elle est ajoutée dans un bucket S3.
- **Point fort** : les ressources nécessaires à l'exécution sont gérées automatiquement par AWS.
- **Point d'attention** : Lambda n'est pas adapté à tous les types d'applications ou de traitements longs et persistants.

À retenir : Lambda permet d'exécuter du code à la demande sans gérer soi-même les machines qui l'exécutent.

Partie 2 – Introduction à Amazon Web Services

5. Identifier les ressources AWS

- Notion de ressource AWS
- ARN – Amazon Resource Name
- Structure générale d'un ARN
- Exemples d'ARN
- Pourquoi les ARN sont importants

Qu'est-ce qu'une ressource AWS ?

Une **ressource AWS** est un élément créé, configuré ou utilisé dans un service AWS.

Quelques exemples :

- une instance **EC2** ;
- un bucket **S3** ;
- une base de données **RDS** ;
- une fonction **Lambda** ;
- un rôle ou un utilisateur **IAM**.

Chaque ressource possède généralement des caractéristiques qui permettent de l'identifier, comme :

- un nom ;
- un identifiant ;
- une Region ;
- un compte AWS ;
- parfois un **ARN**.

À retenir : les services AWS sont les outils proposés par la plateforme ; les ressources sont les éléments concrets que l'on crée ou utilise à l'intérieur de ces services.

ARN – Amazon Resource Name

Un **ARN** (*Amazon Resource Name*) est un identifiant utilisé par AWS pour désigner de manière précise une ressource.

Il permet notamment à AWS de savoir **exactement quelle ressource est concernée** lorsqu'on configure :

- des permissions IAM ;
- des politiques ;
- des règles d'accès ;
- certaines automatisations.

Exemple :

```
arn:aws:s3:::mon-bucket
```

Cet ARN désigne précisément un bucket Amazon S3.

À retenir : un ARN joue le rôle d'un identifiant unique et structuré permettant de référencer une ressource AWS sans ambiguïté.

Structure d'un ARN

Un ARN suit généralement cette structure :

```
arn:partition:service:region:account-id:resource
```

Exemple :

```
arn:aws:lambda:eu-west-3:123456789012:function:ma-fonction
```

On peut y retrouver :

- `arn` → indique qu'il s'agit d'un ARN ;
- `aws` → partition AWS ;
- `lambda` → service concerné ;
- `eu-west-3` → Region ;
- `123456789012` → compte AWS ;
- `function:ma-fonction` → ressource ciblée.

La structure exacte peut varier selon le service.

À retenir : l'ARN décrit généralement le service, la Region, le compte et la ressource ciblée. Il sera particulièrement important lorsqu'on travaillera avec IAM et les permissions.

Partie 2 – Introduction à Amazon Web Services

6. Les différentes façons d'interagir avec AWS

- AWS Management Console
- AWS CLI
- AWS SDK
- SDK AWS pour Python : Boto3
- Comparaison Console / CLI / SDK

AWS Management Console

La **AWS Management Console** est l'interface Web permettant d'accéder aux services AWS depuis un navigateur.

Elle permet notamment de :

- rechercher et ouvrir les services AWS ;
- créer, modifier et supprimer des ressources ;
- consulter l'état des ressources ;
- gérer le compte, la facturation et certains paramètres ;
- sélectionner la **Region** dans laquelle on souhaite travailler.

La Console est particulièrement adaptée pour **découvrir AWS et visualiser les différentes options disponibles**.

À retenir : la Console AWS est l'interface graphique permettant d'administrer AWS depuis un navigateur. Elle est idéale pour apprendre et effectuer des manipulations ponctuelles.

AWS Management Console

The screenshot shows the AWS Management Console 'Console Home' page. Numbered callouts are placed as follows:

- 1**: Points to the user profile dropdown menu in the top right header.
- 2**: Points to the 'Ireland' region selector in the top header.
- 3**: Points to the 'Services' menu icon in the top left header.
- 4**: Points to the search bar in the top header.
- 5**: Points to the 'Actions' dropdown menu in the top right of the main content area.

The main content area includes sections for 'Recently visited' (showing no services), 'Welcome to AWS' (with links for getting started, training, and new services), 'AWS Health' (showing 0 open issues, 0 scheduled changes, and 0 other notifications), 'Cost and usage' (showing current month costs of \$24.32, forecasted month end costs of \$24.32, and last month costs of \$23.64), 'Build a solution', and 'Trusted Advisor'.

AWS CLI

AWS CLI (*Command Line Interface*) permet d'interagir avec AWS directement depuis un terminal. Au lieu d'utiliser des boutons dans la Console, on exécute des commandes.

Exemple :

```
aws s3 ls
```

Cette commande permet de demander à AWS d'afficher les buckets S3 accessibles avec les credentials utilisés.

AWS CLI est particulièrement utile pour :

- répéter rapidement des opérations ;
- automatiser certaines tâches ;
- travailler depuis un terminal ou un script ;
- reproduire précisément une manipulation.

À retenir : AWS CLI permet d'effectuer depuis un terminal une grande partie des opérations disponibles dans la Console AWS.

AWS SDK et Boto3

Un **SDK** (*Software Development Kit*) permet à une application d'interagir directement avec les services AWS depuis du code.

AWS fournit des SDK pour de nombreux langages de programmation.

Pour Python, le SDK utilisé est **Boto3**.

Exemple :

```
import boto3
```

Une application Python peut ensuite utiliser Boto3 pour :

- envoyer un fichier dans S3 ;
- créer ou interroger des ressources AWS ;
- appeler une fonction Lambda ;
- automatiser des opérations selon la logique de l'application.

À retenir : un SDK permet au code d'une application de communiquer directement avec AWS. **Boto3 est le SDK AWS pour Python.**

Exemple Python avec Boto3

Une application Python peut utiliser **Boto3** pour interagir directement avec les services AWS.

Exemple : afficher les buckets S3 accessibles avec les credentials AWS disponibles sur la machine.

```
import boto3

# Création d'un client pour le service Amazon S3.
# Boto3 recherche automatiquement des credentials AWS valides.
s3 = boto3.client("s3")

# Appel de l'API AWS permettant de récupérer la liste des buckets S3
# que l'identité utilisée est autorisée à voir.
response = s3.list_buckets()

# Parcours des buckets retournés par AWS.
for bucket in response["Buckets"]:
    print(bucket["Name"])
```

Console, CLI ou SDK ?

Les trois méthodes permettent d'interagir avec les services AWS, mais répondent à des besoins différents.

Méthode	Utilisation principale
Console AWS	Manipulation graphique, découverte et administration manuelle
AWS CLI	Commandes, scripts et automatisation
AWS SDK	Intégration d'AWS directement dans une application

Dans les trois cas, les actions finissent par solliciter les **API des services AWS**.

Schéma simplifié :

Utilisateur → Console / CLI / SDK → API AWS → Service AWS

À retenir : Console, CLI et SDK ne sont pas trois AWS différents : ce sont trois moyens différents d'interagir avec les mêmes services AWS.

Partie 3 — Premiers fondamentaux de sécurité AWS

Partie 3 – Premiers fondamentaux de sécurité AWS

1. Le compte AWS et les premières bonnes pratiques

- Compte AWS
- Frontière de sécurité et de facturation
- Identifiant du compte AWS
- Compte root
- MFA
- Bonnes pratiques initiales
- Principe du moindre privilège

Le compte AWS : le point de départ

Un **compte AWS** est l'environnement principal dans lequel une organisation ou un utilisateur utilise les services AWS.

Il permet notamment de :

- créer et gérer des **ressources AWS** ;
- gérer des **identités et des permissions** ;
- centraliser la **facturation** liée aux ressources utilisées.

Chaque compte possède un **identifiant unique à 12 chiffres**, appelé **AWS Account ID**.

Exemple :

123456789012

À retenir : le compte AWS constitue le **périmètre principal** dans lequel sont organisées les ressources, les accès et la facturation.

Le compte AWS : une double frontière

Un **compte AWS** constitue deux frontières principales :

- une **frontière de sécurité** : les identités, permissions et ressources sont gérées dans le périmètre du compte ;
- une **frontière de facturation** : les ressources consommées dans le compte sont facturées à ce compte.

Cela signifie qu'un même compte centralise à la fois :

- les ressources AWS
- les accès
- les permissions
- les coûts associés

À retenir : un compte AWS est à la fois un **périmètre de sécurité** et un **périmètre de facturation**.

Le compte root AWS

Le **compte root** est l'identité principale créée lors de l'ouverture du compte AWS. Il possède des privilèges très élevés et peut effectuer des opérations sensibles sur le compte.

Il doit donc être utilisé uniquement lorsque cela est nécessaire.

Bonnes pratiques :

- activer le **MFA** sur le root
- ne pas l'utiliser pour les tâches quotidiennes
- ne jamais créer de **clés d'accès** pour le root
- utiliser des identités dédiées pour l'administration courante.

À retenir : le compte root doit être **fortement protégé** et utilisé le moins possible.

Protéger le compte root avec le MFA

Le **MFA (Multi-Factor Authentication)** ajoute une seconde vérification lors de la connexion.

L'accès repose alors sur plusieurs facteurs, par exemple :

- quelque chose que vous connaissez : le **mot de passe** ;
- quelque chose que vous possédez : une **application d'authentification** ou une **clé de sécurité**.

Même si le mot de passe est compromis, le second facteur limite fortement le risque d'accès non autorisé.

À retenir : le **MFA doit être activé sur le compte root** dès que possible.

Premières bonnes pratiques de sécurité

Dès la création d'un compte AWS, quelques règles doivent être appliquées :

- protéger le **compte root avec le MFA** ;
- ne pas utiliser le root pour les tâches quotidiennes ;
- ne jamais créer de **clés d'accès root** ;
- utiliser des **identités dédiées** pour travailler sur AWS ;
- appliquer le **principe du moindre privilège** ;
- limiter les permissions au strict nécessaire.

À retenir : la sécurité AWS commence par la **protection des identités et des accès**.

Le principe du moindre privilège

Le **principe du moindre privilège** consiste à accorder uniquement les permissions nécessaires pour réaliser une tâche.

Une identité AWS ne doit pas recevoir plus de droits que ce dont elle a réellement besoin.

Exemple :

Un utilisateur qui doit seulement consulter des fichiers dans Amazon S3 n'a pas besoin de pouvoir :

- supprimer des objets ;
- créer des utilisateurs IAM ;
- lancer des instances EC2.

À retenir : moins une identité possède de permissions, plus on limite l'impact potentiel d'une erreur ou d'un compte compromis.

Partie 3 – Premiers fondamentaux de sécurité AWS

2. Premiers réflexes de maîtrise des coûts

- Paiement à l'usage
- AWS Billing
- AWS Free Tier
- AWS Budgets
- Budgets et alertes de coûts
- Coût réel et coût prévisionnel
- Limites des alertes de budget
- Ressources oubliées et contrôle des coûts

Le paiement à l'usage dans AWS

AWS utilise principalement un modèle de **paiement à l'usage** : les coûts dépendent des ressources et services réellement consommés.

La facturation peut notamment dépendre :

- du temps d'utilisation d'une ressource ;
- de la quantité de stockage utilisée ;
- du nombre de requêtes ;
- du volume de données transférées.

Une ressource oubliée peut donc continuer à générer des coûts.

À retenir : dans AWS, il faut surveiller les ressources utilisées et leur coût.

AWS Billing : suivre ses dépenses

AWS Billing and Cost Management regroupe les outils AWS permettant de consulter et gérer les coûts du compte.

- **Catégorie** : gestion financière / coûts
- **Portée** : globale, à l'échelle du compte AWS
- **Utilisation principale** : Console AWS

Il permet notamment de :

- consulter les dépenses ;
- identifier les services qui génèrent des coûts ;
- consulter les factures ;
- suivre l'utilisation des crédits et certaines offres gratuites.

À retenir : contrairement à une ressource EC2 ou RDS, la facturation concerne le **compte AWS dans son ensemble**.

AWS Free Tier

L'**AWS Free Tier** permet aux nouveaux clients de découvrir certains services AWS avec des crédits et des offres gratuites limitées.

Le programme actuel propose notamment :

- **100 USD de crédits** à la création du compte ;
- jusqu'à **100 USD supplémentaires** à obtenir ;
- des offres gratuites sur certains services avec des limites d'utilisation.

Le Free Tier ne signifie pas que **tous les services AWS sont gratuits**.

À retenir : toujours vérifier les conditions et limites du Free Tier avant de créer une ressource.

AWS Budgets

AWS Budgets est un service de gestion des coûts permettant de définir des budgets et de surveiller les dépenses AWS.

- **Catégorie** : gestion financière / coûts
- **Portée** : globale, à l'échelle du compte AWS
- **Utilisation** : principalement Console AWS, mais également API et AWS CLI

Exemple :

```
Budget mensuel : 10 USD
    ↓
Seuil 50 %
Seuil 80 %
Seuil 100 %
    ↓
Notifications
```

Un budget permet donc d'être averti lorsque les dépenses approchent ou dépassent certains seuils.

À retenir : AWS Budgets est un outil de surveillance, pas simplement un affichage de facture.

Coût réel et coût prévisionnel

AWS Budgets peut surveiller deux types de valeurs :

- **coût réel** : dépenses déjà enregistrées ;
- **coût prévisionnel** : estimation des dépenses futures à partir de l'utilisation observée.

Une alerte peut être déclenchée lorsqu'un seuil défini est atteint ou risque de l'être.

Attention : les prévisions nécessitent suffisamment de données historiques pour être calculées.

À retenir : surveiller le réel permet de constater, surveiller le prévisionnel permet d'anticiper.

Budgets : limites et bonnes pratiques

Un budget et ses alertes ne remplacent pas une bonne gestion des ressources.

Il faut notamment :

- définir des alertes suffisamment tôt ;
- surveiller régulièrement Billing ;
- vérifier les ressources encore actives ;
- supprimer les ressources devenues inutiles ;
- ne jamais supposer qu'une ressource est gratuite.

Une alerte AWS Budgets **n'empêche pas automatiquement une dépense** : elle sert avant tout à prévenir.

À retenir : la meilleure protection contre les coûts inattendus reste la **surveillance et le nettoyage régulier des ressources**.

Partie 3 – Premiers fondamentaux de sécurité AWS

3. Introduction à AWS IAM

- AWS IAM : le service qui contrôle les accès
- Authentification et autorisation
- Identités et permissions
- Principe du moindre privilège

AWS IAM : le service qui contrôle les accès

AWS IAM (Identity and Access Management) est le service AWS utilisé pour gérer les identités et les permissions.

- **Catégorie** : sécurité, identité et conformité
- **Portée** : globale
- **Utilisation** : Console AWS, AWS CLI, SDK et API

IAM permet de contrôler :

- qui peut accéder à AWS ;
- quelles actions peuvent être réalisées ;
- sur quelles ressources.

Pour découvrir et configurer IAM, la **Console AWS** est souvent la plus lisible au début. La **CLI et les SDK** deviennent ensuite utiles pour automatiser et intégrer ces opérations. À retenir : IAM est le service central de gestion des accès dans AWS.

Authentification et autorisation

IAM intervient dans deux étapes différentes :

Authentification

Vérifier **qui est l'identité** qui tente d'accéder à AWS.

Exemples :

- mot de passe ;
- MFA ;
- credentials AWS.

Autorisation

Déterminer **ce que cette identité a le droit de faire**.

```
Authentification → Qui êtes-vous ?  
Autorisation      → Que pouvez-vous faire ?
```

À retenir : être authentifié ne signifie pas automatiquement être autorisé à effectuer une action.

Identités et permissions

Dans AWS, une **identité** représente un utilisateur, une application ou un service qui doit effectuer des actions.

Les **permissions** définissent ensuite ce que cette identité est autorisée à faire.

Exemple :

```
Identité
  ↓
Permissions
  ↓
s3:GetObject
  ↓
Bucket S3
```

Une identité peut donc être reconnue par AWS tout en ayant des permissions très limitées.

À retenir : IAM sépare l'identité de ses droits d'accès.

Rappel – Principe du moindre privilège

Comme vu précédemment, le **moindre privilège** consiste à accorder uniquement les permissions nécessaires.

Avec IAM, cela signifie par exemple :

- autoriser uniquement les actions utiles
- limiter l'accès aux ressources nécessaires
- éviter les permissions trop larges comme `*` lorsque ce n'est pas justifié.

Exemple :

```
Besoin : lire des fichiers S3  
→ Autoriser la lecture  
→ Ne pas autoriser la suppression ou l'administration IAM
```

À retenir : avec IAM, les permissions doivent être aussi limitées que possible tout en permettant la tâche attendue.

Partie 3 – Premiers fondamentaux de sécurité AWS

4. Utilisateurs, groupes et rôles IAM

- IAM User
- IAM Group
- IAM Role
- Différences entre User, Group et Role
- Relations entre humains, applications et services AWS

IAM User

Un **IAM User** est une identité créée dans AWS IAM.

Il peut représenter une personne ou, dans certains cas, un usage nécessitant une identité dédiée.

Un IAM User :

- appartient à un seul compte AWS
- peut disposer d'un accès à la Console AWS ou d'un accès programmatique
- ne possède **aucune permission par défaut**
- reçoit ses permissions grâce aux politiques IAM.

À retenir : créer un IAM User ne lui donne pas automatiquement accès aux ressources AWS.

IAM Group

Un **IAM Group** permet de regrouper plusieurs IAM Users afin de gérer leurs permissions plus facilement.

Une policy peut être attachée à un groupe, puis ses permissions sont appliquées aux utilisateurs membres.

Policy → IAM Group → IAM Users

Un utilisateur peut appartenir à **plusieurs groupes**.

À retenir : les groupes simplifient la gestion des permissions lorsque plusieurs utilisateurs ont les mêmes besoins.

IAM Role

Un **IAM Role** est une identité IAM conçue pour être **assumée temporairement**. Contrairement à un IAM User, un rôle :

- n'est pas associé à une personne précise ;
- ne possède pas de credentials permanents propres ;
- peut être utilisé par une identité, une application ou un service AWS.

Exemple :

Application sur EC2 → IAM Role → Amazon S3

À retenir : les rôles permettent d'accorder des permissions sans stocker de credentials permanents dans l'application.

IAM User, Group et Role : quelles différences ?

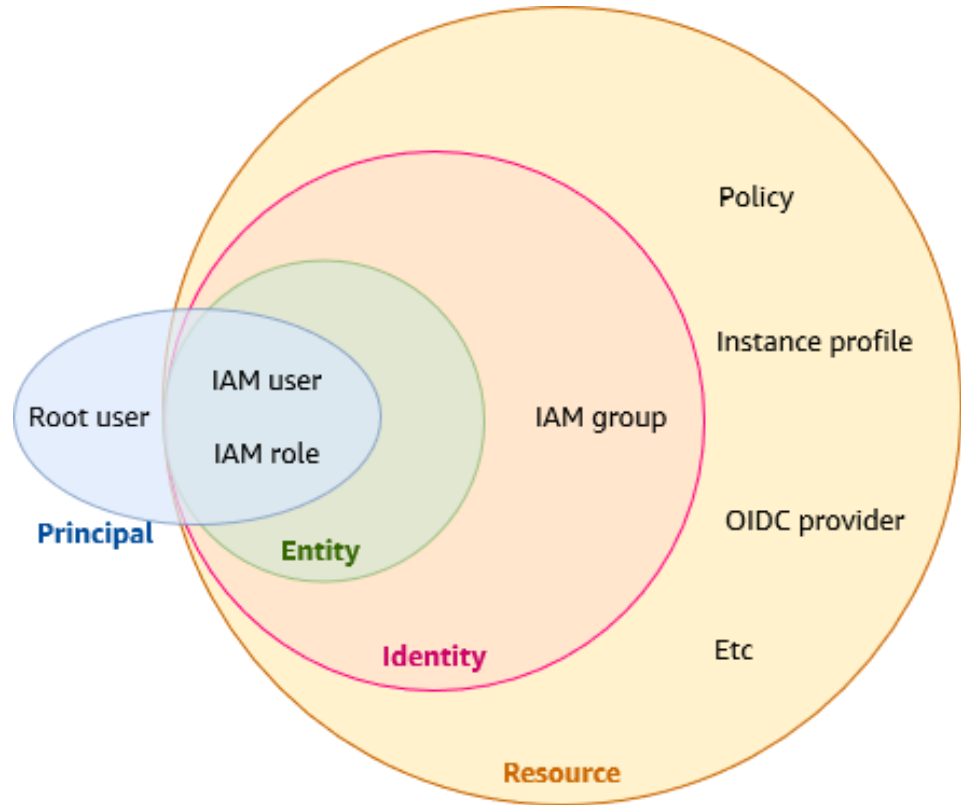
Élément	Rôle principal
IAM User	Représenter une identité créée dans le compte
IAM Group	Regrouper des utilisateurs et leurs permissions
IAM Role	Fournir temporairement des permissions à une identité, une application ou un service

Un **Group** n'est pas une identité : il sert à organiser des IAM Users.

Un **Role** est une identité qui peut être assumée temporairement.

À retenir : User, Group et Role répondent à des besoins différents et complémentaires.

IAM User, Group et Role



[docs aws iam](#)

[docs aws guide utilisateur](#)

Comment les identités IAM obtiennent-elles des permissions ?

IAM permet de séparer les **identités** des **permissions** qui leur sont accordées.

Pour un utilisateur :

Humain → IAM User → IAM Group → Politiques → Ressources AWS

Pour une application ou un service :

Application / Service AWS → IAM Role → Politiques → Ressources AWS

Les **politiques IAM** déterminent précisément les actions autorisées ou refusées.

À retenir : les identités indiquent **qui agit**, les politiques déterminent **ce qui est permis**.

Partie 3 – Premiers fondamentaux de sécurité AWS

5. Politiques IAM

- Policy et document JSON
- Version
- Statement
- Sid
- Effect
- Action
- Resource
- Principal
- Condition
- Allow et Deny
- Deny implicite et Deny explicite
- Priorité du Deny explicite
- Wildcards
- Introduction à NotAction

Qu'est-ce qu'une policy IAM ?

Une **IAM Policy** est un document qui définit des permissions dans AWS.

Elle indique notamment :

- quelles **actions** sont autorisées ou refusées ;
- sur quelles **ressources** ;
- éventuellement dans quelles **conditions**.

Les policies sont écrites au format **JSON**.

Elles peuvent être associées à des identités IAM ou, selon le service, directement à certaines ressources.

À retenir : une policy traduit les règles d'autorisation appliquées dans AWS.

Structure générale d'une policy IAM

Une policy IAM contient généralement une ou plusieurs règles appelées **Statements**.

Exemple :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LectureS3",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mon-bucket/*"
    }
  ]
}
```

- **Version** : version du langage de policy utilisé (ce n'est pas la date d'écriture)
- **Statement** : ensemble des règles de permission
- **Sid** : identifiant optionnel permettant de nommer une règle.

À retenir : une policy peut contenir plusieurs Statements évalués séparément.

Effect, Action et Resource

Trois éléments permettent de comprendre l'essentiel d'un Statement.

Effect

Indique le résultat de la règle :

- `Allow` : autoriser ;
- `Deny` : refuser.

Action

Indique l'opération concernée.

Exemple : `s3:GetObject`

Resource

Indique la ressource AWS concernée, généralement grâce à son **ARN**.

Exemple : `arn:aws:s3:::mon-bucket/*`

À retenir : une règle répond essentiellement à la question :
autoriser ou refuser quelle action, sur quelle ressource ?

Principal et Condition

Certaines politiques utilisent aussi les éléments **Principal** et **Condition**.

Principal

Indique **quelle identité** est concernée par la règle.

Exemples :

- un compte AWS ;
- un IAM User ;
- un IAM Role ;
- un service AWS.

Principal est notamment utilisé dans les **resource-based policies** et les politiques de confiance des rôles.

Condition

Permet d'appliquer une règle uniquement dans certaines circonstances.

Exemples :

- région utilisée ;
- présence du MFA ;
- adresse IP source.

À retenir : **Principal** précise **qui**, tandis que **Condition** précise **dans quel contexte**.

Allow, Deny et comportement par défaut

Dans IAM, une action n'est pas automatiquement autorisée.

Trois situations sont importantes :

- **Allow explicite** : une policy autorise l'action ;
- **Deny explicite** : une policy refuse explicitement l'action ;
- **Deny implicite** : aucune policy n'autorise l'action.

Par défaut, une identité ne peut donc effectuer que les actions qui lui ont été explicitement autorisées.

À retenir : **absence d'autorisation = refus.**

Comment AWS évalue-t-il les permissions ?

Lorsqu'une identité tente d'effectuer une action, AWS évalue les politiques applicables.

Le principe général est :

Deny explicite → Refus

Sinon, Allow explicite → Autorisation

Sinon → Deny implicite

Un Deny explicite est prioritaire sur un Allow.

Exemple :

Une policy autorise la lecture d'un bucket S3, mais une autre refuse explicitement l'accès à un dossier sensible.

L'accès à ce dossier sera refusé.

À retenir : **un Deny explicite l'emporte sur un Allow.**

Wildcards et NotAction

Les politiques IAM permettent d'utiliser des expressions plus larges.

Wildcard *

Le caractère * permet de représenter plusieurs actions ou ressources.

Exemples :

- s3:* → toutes les actions S3 ;
- Resource: "*" → toutes les ressources concernées.

Ces expressions doivent être utilisées avec prudence car elles peuvent accorder des permissions très larges.

NotAction

NotAction permet de définir une règle concernant **toutes les actions sauf celles indiquées**.

Cette syntaxe est utile dans certains cas, mais peut être plus difficile à lire et doit être utilisée avec précaution.

À retenir : les wildcards et NotAction sont puissants, mais peuvent facilement élargir les permissions au-delà du besoin réel.

Liens utiles – IAM Policies

AWS Policy Generator

Créer une policy étape par étape et générer automatiquement son JSON.

<https://awspolicygen.s3.amazonaws.com/policygen.html>

AWS Service Authorization Reference

Référence officielle pour connaître, service par service :

- les actions IAM disponibles ;
- leur rôle ;
- les ressources compatibles ;
- les clés de condition utilisables.

https://docs.aws.amazon.com/service-authorization/latest/reference/reference_policies_actions-resources-contextkeys.html

Exemples :

- Amazon S3 : `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`, etc.
- Amazon EC2 : `ec2:RunInstances`, `ec2:StopInstances`, `ec2:DescribeInstances`, etc.

Partie 3 — Premiers fondamentaux de sécurité AWS

6. Types de politiques IAM

- Identity-based policies
- Resource-based policies
- AWS managed policies
- Customer managed policies
- Inline policies

Deux façons de classer les policies IAM

Les policies IAM peuvent être classées selon deux questions différentes.

Où la policy est-elle appliquée ?

- à une **identité** ;
- directement à une **ressource AWS**.

Qui crée et gère la policy ?

- AWS ;
- le client AWS ;
- directement dans une identité ou une ressource.

Ces deux classifications sont différentes et complémentaires.

À retenir : une policy peut être décrite à la fois par **son emplacement** et par **son mode de gestion**.

Identity-based policies

Une **identity-based policy** est attachée à une identité IAM :

- IAM User ;
- IAM Group ;
- IAM Role.

Elle définit les actions que cette identité peut effectuer sur les ressources AWS.

Exemple :

IAM Role → Policy → Autorisation de lire des objets dans S3

Les permissions accordées dépendent du contenu de la policy.

À retenir : une identity-based policy répond principalement à la question :

« Que cette identité a-t-elle le droit de faire ? »

Resource-based policies

Une **resource-based policy** est attachée directement à une ressource AWS compatible.

Elle permet notamment de définir **qui peut accéder à cette ressource**.

Exemple courant :

Bucket Amazon S3 → Bucket Policy

Dans ce type de policy, le champ `Principal` permet d'indiquer l'identité concernée.

Toutes les ressources AWS ne prennent pas en charge les resource-based policies.

À retenir : une resource-based policy définit les règles d'accès directement depuis la ressource.

AWS managed, Customer managed et Inline policies

AWS managed policy

Policy créée et maintenue par AWS.

Exemples : `ReadOnlyAccess`, `AmazonS3ReadOnlyAccess`

Customer managed policy

Policy créée et maintenue dans votre compte AWS.

Elle peut être réutilisée par plusieurs identités.

Inline policy

Policy intégrée directement dans une seule identité ou ressource.

Elle est fortement liée à cet élément et n'est pas conçue pour être réutilisée.

À retenir : ces trois types décrivent **qui gère la policy et comment elle est réutilisée**.

Quelle approche choisir ?

Le choix dépend du besoin.

Type	Utilisation principale
AWS managed	Permissions génériques prêtes à l'emploi
Customer managed	Permissions personnalisées et réutilisables
Inline	Besoin très spécifique lié à un seul élément

En pratique :

- privilégier des permissions adaptées au besoin réel ;
- éviter les politiques trop larges ;
- préférer des politiques réutilisables lorsqu'elles doivent être partagées ;
- appliquer le **principe du moindre privilège**.

À retenir : le meilleur choix est celui qui reste **clair, maîtrisé et suffisamment restrictif**.

Partie 3 – Premiers fondamentaux de sécurité AWS

7. Credentials AWS

- Credentials pour la Console
- Access Key ID
- Secret Access Key
- Credentials permanents
- Credentials temporaires
- Session Token
- Introduction à AWS STS
- Credentials et IAM Roles

Qu'est-ce qu'un credential AWS ?

Un **credential AWS** est un élément utilisé pour prouver une identité lors d'un accès à AWS.

Selon le mode d'accès, les credentials peuvent être différents :

- **Console AWS** : mot de passe, MFA ou mécanisme fédéré
- **AWS CLI / SDK / API** : credentials d'accès programmatiques.

Les credentials servent à l'**authentification**.

Ils ne définissent pas directement les permissions : celles-ci sont déterminées ensuite par IAM.

À retenir : les credentials permettent de prouver **qui vous êtes**, pas **ce que vous avez le droit de faire**.

Access Key ID et Secret Access Key

Pour un accès programmatique, AWS peut utiliser une paire de clés :

- **Access Key ID** : identifiant de la clé ;
- **Secret Access Key** : secret associé à cette clé.

Exemple :

Access Key ID + Secret Access Key → authentication auprès des API AWS

La Secret Access Key doit rester strictement confidentielle.

Elle ne doit jamais être :

- publiée dans un dépôt Git ;
- écrite en dur dans le code ;
- intégrée dans une image Docker ;
- partagée avec d'autres utilisateurs.

À retenir : une Secret Access Key doit être protégée comme un mot de passe.

Credentials permanents et temporaires

AWS distingue deux grandes catégories de credentials.

Credentials longue durée

Ils restent valides tant qu'ils ne sont pas désactivés ou supprimés.

Exemple :

- Access Key ID ;
- Secret Access Key.

Credentials temporaires

Ils possèdent une durée de vie limitée et expirent automatiquement.

Ils contiennent généralement :

- Access Key ID ;
- Secret Access Key ;
- **Session Token.**

À retenir : lorsque c'est possible, les credentials temporaires sont préférables aux clés longue durée.

AWS STS et les IAM Roles

AWS STS (Security Token Service) est le service AWS utilisé pour fournir des **credentials temporaires**.

- **Catégorie** : sécurité et identité
- **Portée** : service global
- **Utilisation** : API, AWS CLI et SDK

Un IAM Role peut être assumé par une identité, une application ou un service AWS.

Identité / Application → IAM Role → AWS STS → Credentials temporaires

Ces credentials permettent ensuite d'utiliser les permissions associées au rôle.

À retenir : les IAM Roles et STS permettent d'accéder à AWS sans dépendre de credentials permanents.

Bonnes pratiques pour les credentials AWS

Quelques règles essentielles :

- éviter les **clés d'accès longue durée** lorsque possible ;
- privilégier les **IAM Roles** et les credentials temporaires ;
- ne jamais stocker de secrets dans le code ou dans Git ;
- ne jamais créer de clés d'accès pour le compte root ;
- supprimer ou désactiver les credentials devenus inutiles ;
- ne pas partager les credentials entre plusieurs utilisateurs.

Pour les applications exécutées dans AWS, privilégier un **IAM Role** plutôt que des clés stockées dans la configuration.

À retenir : moins les secrets sont permanents et exposés, plus le risque de compromission est réduit.

Partie 3 – Premiers fondamentaux de sécurité AWS

8. Premières bonnes pratiques IAM

- Ne pas utiliser le compte root au quotidien
- Activer le MFA
- Appliquer le moindre privilège
- Éviter les permissions trop larges
- Éviter les clés d'accès longue durée lorsque possible
- Utiliser les rôles IAM
- Privilégier les credentials temporaires
- Ne jamais stocker de secrets dans le code ou Git

Bonnes pratiques IAM – Sécuriser les identités humaines

Rappel : plusieurs de ces règles ont déjà été vues lors de l'étude du compte root et du MFA.

Pour les accès humains :

- ne pas utiliser le **compte root** au quotidien
- protéger les comptes avec le **MFA**
- éviter les comptes partagés
- utiliser une identité propre à chaque utilisateur
- privilégier des mécanismes d'accès temporaires ou fédérés lorsque possible.

À retenir : chaque utilisateur doit disposer d'un accès **identifiable, protégé et limité**.

Bonnes pratiques IAM – Limiter les permissions

Rappel : le principe du **moindre privilège** consiste à accorder uniquement les permissions nécessaires.

Il faut notamment :

- éviter les permissions trop larges
- limiter les actions aux besoins réels
- limiter l'accès aux ressources concernées
- éviter les wildcards * lorsqu'elles ne sont pas nécessaires
- revoir régulièrement les permissions accordées.

À retenir : une permission inutile augmente inutilement le risque en cas d'erreur ou de compromission.

Bonnes pratiques IAM – Accès programmatiques

Rappel : les credentials temporaires sont préférables aux clés d'accès longue durée lorsque c'est possible.

Pour les applications et services :

- privilégier les **IAM Roles** ;
- utiliser des **credentials temporaires**
- éviter de créer des Access Keys permanentes sans nécessité
- ne jamais stocker de secrets dans le code
- ne jamais publier de credentials dans Git ou une image Docker.

À retenir : une application AWS ne devrait pas dépendre de secrets permanents lorsqu'un rôle IAM peut être utilisé.

Bonnes pratiques IAM – Nettoyer les accès

La sécurité IAM ne s'arrête pas à la création des identités et des permissions.

Il faut régulièrement :

- supprimer les identités devenues inutiles
- désactiver ou supprimer les clés d'accès inutilisées
- retirer les permissions qui ne sont plus nécessaires
- vérifier les rôles et policies encore utilisés
- maintenir les droits selon le principe du moindre privilège.

À retenir : les accès AWS doivent évoluer avec les besoins et être régulièrement réévalués.

Partie 3 — Premiers fondamentaux de sécurité AWS

9. Premiers fondamentaux AWS CLI

- AWS CLI v2
- Installation et vérification
- Configuration
- Fichiers ~/.aws/config et ~/.aws/credentials
- Profils AWS CLI
- --profile
- AWS_PROFILE
- Région par défaut et --region
- aws sts get-caller-identity
- Formats de sortie avec --output
- Introduction à --query et JMESPath

AWS CLI v2 : installation

AWS CLI permet d'interagir avec AWS directement depuis un terminal.

Pour installer la version actuelle :

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

AWS fournit des procédures pour :

- Windows ;
- macOS ;
- Linux.

Vérifier ensuite l'installation :

```
aws --version
```

À retenir : privilégier **AWS CLI v2**, qui est la version actuelle recommandée par AWS.

AWS CLI v2 : utiliser AWS depuis un terminal

AWS CLI (Command Line Interface) permet d'interagir avec AWS depuis un terminal. Comme vu précédemment, la Console, la CLI et les SDK utilisent les **API AWS**.

AWS CLI permet notamment de :

- consulter des ressources ;
- créer ou modifier des ressources ;
- automatiser des opérations ;
- reproduire facilement une même action.

Vérifier l'installation :

```
aws --version
```

À retenir : la CLI permet de piloter AWS sans passer par l'interface graphique.

Configurer AWS CLI

AWS CLI doit connaître plusieurs informations pour fonctionner :

- les credentials utilisés ;
- la région par défaut ;
- le format de sortie souhaité.

La commande suivante permet de configurer ces éléments :

```
aws configure
```

Exemple de paramètres :

- région : `eu-west-3`
- format : `json`

Rappel : les credentials permettent à la CLI de s'authentifier auprès d'AWS.

À retenir : la configuration permet à AWS CLI de savoir avec quelle identité et dans quel contexte travailler.

Les fichiers de configuration AWS CLI

AWS CLI utilise principalement deux fichiers dans le répertoire utilisateur.

`~/.aws/credentials`

Contient les credentials configurés pour les profils.

`~/.aws/config`

Contient principalement :

- la région par défaut ;
- le format de sortie ;
- d'autres paramètres de configuration.

Le fichier `credentials` contient des informations sensibles et doit être protégé.

À retenir : `credentials` stocke les secrets, `config` stocke principalement les préférences.

Les profils AWS CLI

Un **profil AWS CLI** regroupe une configuration utilisable sous un nom.

AWS CLI peut utiliser :

- un profil `default` ;
- plusieurs profils nommés.

Exemple :

```
aws sts get-caller-identity --profile formation
```

Il est aussi possible de sélectionner un profil pour toute la session :

```
export AWS_PROFILE=formation
```

Les profils sont particulièrement utiles lorsqu'on travaille avec plusieurs comptes ou identités AWS.

À retenir : toujours savoir **quel profil** est utilisé avant d'exécuter une commande importante.

Vérifier son identité et sa région

Avant une opération importante, il est utile de vérifier l'identité utilisée par AWS CLI.

Commande essentielle :

```
aws sts get-caller-identity
```

Elle permet notamment d'obtenir :

- l'identifiant du compte AWS ;
- l'identité utilisée ;
- son ARN.

La région peut être définie dans le profil ou spécifiée ponctuellement :

```
--region eu-west-3
```

À retenir : vérifier **l'identité, le compte et la région** permet d'éviter de nombreuses erreurs.

Filtrer et formater les résultats AWS CLI

Les commandes AWS CLI retournent souvent beaucoup d'informations.

--output

Permet de choisir le format d'affichage :

- json
- table
- text

Exemple :

```
aws ec2 describe-regions --output table
```

--query

Permet de filtrer les données retournées grâce au langage **JMESPath**.

Exemple :

```
aws ec2 describe-regions --query "Regions[].RegionName"
```

À retenir : **--output** améliore l'affichage, tandis que **--query** permet de sélectionner les informations utiles.

AWS CLI – Références utiles

Documentation des commandes

<https://docs.aws.amazon.com/cli/latest/reference/>

Permet de retrouver toutes les commandes AWS CLI et leurs options.

Configuration et profils

<https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html>

Cette documentation explique notamment :

- le profil `default` ;
- les profils nommés ;
- les fichiers `config` et `credentials` ;
- comment sélectionner un profil ;
- comment modifier sa configuration.

Commandes utiles

```
aws configure --profile formation
```

```
aws configure list-profiles
```

```
aws sts get-caller-identity --profile formation
```

Questions ?

